SIM ID: 10263704

NAME: Subathra Sundarbabu

DATE: 7 Jan 2026

CM060 Natural Language Processing

Fake News Detection using Statistical and Embedding Models

Introduction

1. Domain-Specific Area

1.1 Problem Context

Natural Language Processing (NLP), focuses on identifying false or misleading information presented in the form of true news, fake news detection is a major and difficult problem. Real and fake news can now spread at previously unimaginable speeds and scales due to the rapid growth of social media and online news platforms. Oshikawa, Qian and Wang define automated fake news detection as “the task of assessing the truthfulness of claims in news” (Oshikawa et al., 2020), pointing out its importance as a current NLP issue. Fake news, in contrast to traditional misinformation, is often intentionally created to mimic reliable journalism, making it challenging for readers to differentiate between real and fake content using only their intuition.

1.2 Societal and Industry Impact

Fake news has a major impact on society and business. Misinformation can affect political decisions, lower public confidence in media organisations, and cause instability in society, according to research. Ferguson and Avornu emphasise that fake news has “the ability to cause massive social and national damage with devastating effects in this digital era” (Ferguson and Avornu, 2022), especially when social media platforms enhance it. Although accurate, manual fact-checking by journalists and experts is expensive, time-consuming, and unable to keep up with the huge amount of digital content created every day. Because of this, automated methods are becoming more and more necessary to support journalism, policy enforcement, and content moderation.

1.3 Role of Text Classification

Text classification techniques provide a systematic and flexible answer to this problem. NLP-based classifiers can help automatically detect false or misleading content by analysing word usage, language patterns, and semantic structures in news articles. Although they have different advantages and disadvantages, earlier research shows that both conventional machine learning methods and modern deep learning models can significantly help in the detection of fake news (Oshikawa et al., 2020). As a result, the detection of fake news is a significant application area where developments in text classification can directly address a problem of society in the real world.

2. Objectives

2.1 Goal

This project's main goal is to look into and compare the effectiveness and usefulness of embedding-based deep learning models and traditional statistical text classification models in the context of fake news detection. Built-in textual features like bag-of-words or TF-IDF representations are commonly used in statistical methods like Naïve Bayes, Logistic Regression, and Support Vector Machines. On the other hand, complex contextual representations of language that capture syntax and semantic relationships within text are learned by embedding-based models, such as neural networks. For well-informed model selection, it is important to understand how these simply different approaches perform on the same classification task.

Embedding-based models often achieve better predictive performance, according to previous studies. Prachi et al. report that while classical machine learning models achieve moderate accuracy, “the NLP-based BERT technique obtained the highest accuracy of 98%” (Prachi et al., 2022). Similarly, Roumeliotis, Tselikas and Nasiopoulos observe that advanced embedding-based models “significantly outperform traditional models” in fake news classification tasks, despite the fact that they also observe higher computational complexity and costs (Roumeliotis et al., 2025). These results show that understanding and effectiveness of resources might be affected as a result of increased accuracy.

2.2 Potential Contributions

By offering empirical proof that good prediction accuracy can be attained without necessarily depending on extremely complicated embedding-based models, the study's findings contribute to the difficulty of detecting fake news. Although previous studies have often demonstrated higher accuracy using transformer-based and deep learning approaches (Prachi et al., 2022), these studies also recognise that these models have higher processing costs and less transparency.

The project's results confirm previous findings that, in some text categorisation scenarios, conventional feature-based methods are still competitive. Oshikawa, Qian, and Wang's

(2020) survey study tells that, especially when datasets are well selected, more straightforward lexical and statistical representations can capture powerful signals for truth assessment. In similar, Probierz et al. (2021) show that traditional machine learning methods can be used to detect fake news effectively while emphasising speed and effectiveness.

This project provides evidence that simpler and easier-to-understand models could provide a practical and resource-efficient substitute for detecting fake news, particularly in real-world deployment scenarios where explainability, cost and scalability are important factors.

3. Dataset Description

This project used a dataset that was chosen to be accurate for the fake news detection problem and to support all the steps in the text classification pipeline that were described in this coursework. To find fake news, need datasets that have real and fake news articles with clear labels, enough text, and a range of real-world situations. The Fake and Real News dataset meets these needs by giving labelled news articles that can be used for supervised learning and comparing different classification methods. Similar datasets have been widely used in earlier research, where fake news detection depends on recognising linguistic and semantic patterns between true and fake news content (Oshikawa et al., 2020). The dataset is therefore appropriate for exploring both statistical and embedding-based text classification methods.

3.1 Size

Analysis of the dataset shows that Fake.csv contains 23,481 news articles, while True.csv contains 21,417 articles, which results in a combined total of 44,898 samples. The relatively balanced distribution between fake and real news classes decreases the risk of severe class inequality and supports accurate supervised learning and evaluation. The dataset size is sufficiently large to allow robust training of both traditional machine learning models and neural network–based classifiers, while still practical for experimentation.

In [1]:
```
# Import the required library
import pandas as pd
```

In [2]:
```
# Read the 2 different files in to the dataframe seperately
df_fake = pd.read_csv("dataset/Fake.csv")
df_true = pd.read_csv("dataset/True.csv")
```

In [3]:
```
# Size of the different dataframe
print(df_fake.shape)
print(df_true.shape)
```

```
(23481, 4)
(21417, 4)
```

In [4]:
```
# first 5 rows in df_fake
df_fake.head()
```

Out[4]:

	title	text	subject	date
0	Donald Trump Sends Out Embarrassing New Year'...	Donald Trump just couldn t wish all Americans ...	News	December 31, 2017
1	Drunk Bragging Trump Staffer Started Russian ...	House Intelligence Committee Chairman Devin Nu...	News	December 31, 2017
2	Sheriff David Clarke Becomes An Internet Joke...	On Friday, it was revealed that former Milwauk...	News	December 30, 2017
3	Trump Is So Obsessed He Even Has Obama's Name...	On Christmas day, Donald Trump announced that ...	News	December 29, 2017
4	Pope Francis Just Called Out Donald Trump Dur...	Pope Francis used his annual Christmas Day mes...	News	December 25, 2017

In [5]:
```
# first 5 rows in df_true
df_true.head()
```

Out[5]:

	title	text	subject	date
0	As U.S. budget fight looms, Republicans flip t...	WASHINGTON (Reuters) - The head of a conservat...	politicsNews	December 31, 2017
1	U.S. military to accept transgender recruits o...	WASHINGTON (Reuters) - Transgender people will...	politicsNews	December 29, 2017
2	Senior U.S. Republican senator: 'Let Mr. Muell...	WASHINGTON (Reuters) - The special counsel inv...	politicsNews	December 31, 2017
3	FBI Russia probe helped by Australian diplomat...	WASHINGTON (Reuters) - Trump campaign adviser ...	politicsNews	December 30, 2017
4	Trump wants Postal Service to charge 'much mor...	SEATTLE/WASHINGTON (Reuters) - President Donal...	politicsNews	December 29, 2017

3.2 Data Types

All four columns are saved as object data types. When evaluated using normal missing-value checks, both the real and fake news data had no null values. Additional investigation shows that some text fields contain empty string values, which are not seen as missing data since they are saved as string objects rather than nulls. Large, automatically gathered text corpora frequently include this feature, which has no impact on the dataset's general usefulness for natural language processing. In order to ensure data quality before feature extraction and model training, these situations were handled during preprocessing. All text fields were first same by removing leading and trailing whitespace in order to address these situations. Following this stage, entries with zero-length text were clearly marked as invalid records.

Since empty text provides no useful information for feature extraction or model training, these samples were later removed from the dataset. Before tokenization, vectorisation, and model evaluation, this preprocessing step made sure that only textual data was kept.

In [6]:
```
# view data types for the df_true
df_true.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 21417 entries, 0 to 21416
Data columns (total 4 columns):
 #   Column   Non-Null Count  Dtype 
---  ------   --------------  ----- 
 0   title    21417 non-null  object
 1   text     21417 non-null  object
 2   subject  21417 non-null  object
 3   date     21417 non-null  object
dtypes: object(4)
memory usage: 669.4+ KB
```

In [7]:
```
# view data types for df_fake
df_fake.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 23481 entries, 0 to 23480
Data columns (total 4 columns):
 #   Column   Non-Null Count  Dtype 
---  ------   --------------  ----- 
 0   title    23481 non-null  object
 1   text     23481 non-null  object
 2   subject  23481 non-null  object
 3   date     23481 non-null  object
dtypes: object(4)
memory usage: 733.9+ KB
```

In [8]:
```
# check empty string values in df_true
(df_true.apply(lambda col: col.astype(str).str.strip() == "")).sum()
```

Out[8]:
```
title      0
text       1
subject    0
date       0
dtype: int64
```

In [9]:
```
# check empty string values in df_fake
(df_fake.apply(lambda col: col.astype(str).str.strip() == "")).sum()
```

Out[9]:
```
title        0
text       630
subject      0
date         0
dtype: int64
```

df_true has 1 empty string in the text column. df_fake have 630 empty string in the text column.

3.3 Data Origin

The dataset was obtained from Kaggle. Datasets collected from various sources, such as reputable news outlets and websites are hosted on Kaggle. Previous studies have made considerable use of comparable datasets, where fake news content is gathered from websites that have been labeled as fake, while true news stories are supplied from reliable publications like Reuters (Probierz et al., 2021). Its origin promotes repeatability and comparability with current studies on fake news identification because it is a publicly accessible and thoroughly documented dataset.

4. Evaluation Methodology

4.1 Description of Metrics

This project uses measures that are frequently used in text classification and false news detection research to evaluate and compare the performance of statistical and embedding-based models. These measurements include F1-score, recall, accuracy, and precision. Accuracy gives an overview of the model's performance by calculating the total percentage of correctly classified data. But accuracy might not be enough on its own, especially when it comes to fake news detection tasks where misclassification might have a significant cost.

Recall and precision provide a more complex understanding of classifier behaviour. Recall represents the percentage of real fake news data that are successfully identified, whereas precision indicates the percentage of correctly identified fake news articles among all articles predicted as fake. When assessing binary classification problems, the F1-score which is the mean of precision and recall offers a balanced metric that is very helpful. Prachi et al. highlight the importance of multi-metric evaluation, noting that their models were assessed using “accuracy, precision, recall, F-1 score, ROC curve, etc.” to ensure comprehensive performance analysis (Prachi et al., 2022).

4.2 Application of metrics to compare two methodologies

By capturing both predicted accuracy and error characteristics, these metrics allow statistical and embedding-based techniques to be fairly compared. Because of the social effects of false positives and false negatives, Oshikawa et al. highlight that fake news detection is usually structured as a classification task where thorough judgment is required (Oshikawa et al., 2020). This research ensures consistency and makes it possible to draw conclusions about the relative effectiveness and applicability of both model types by using the same evaluation approach.

Implementation

5. Data Processing

This section explains the conversion and local storage of the fake news dataset, as well as the preprocessing steps used to get the text ready for modeling. Additionally, it describes the text representations that were selected and highlights how statistical frequency-based models and embedding-based models require different data preparation. These procedures ensure data quality, reproducibility, and fair comparisons between modeling techniques.

5.1 Assigning Labels and Merging

The raw datasets were first loaded, labeled, combined, and locally saved to ensure consistency and ensure similar preprocessing. A single dataset with an binary label was created by combining the real and fake news files, giving all subsequent evaluations a single source.

In [10]:
```
# Import the required library
import pandas as pd
import re
```

In [11]:
```
# Assign the binary labels for fake and true
df_fake["label"] = 0
df_true["label"] = 1
```

Real news articles are stored as class 1 in this implementation, while fake news articles are encoded as class 0. The positive class (class 1) is represented by all reported class-specific precision, recall, and F1-scores.

In [12]:
```
# Merge the dataset
df = pd.concat([df_true, df_fake], ignore_index=True)
```

In [13]:
```
# view the first 5 rows of the df
df.head()
```

Out[13]:

	title	text	subject	date	label
0	As U.S. budget fight looms, Republicans flip t...	WASHINGTON (Reuters) - The head of a conservat...	politicsNews	December 31, 2017	1
1	U.S. military to accept transgender recruits o...	WASHINGTON (Reuters) - Transgender people will...	politicsNews	December 29, 2017	1
2	Senior U.S. Republican senator: 'Let Mr. Muell...	WASHINGTON (Reuters) - The special counsel inv...	politicsNews	December 31, 2017	1
3	FBI Russia probe helped by Australian diplomat...	WASHINGTON (Reuters) - Trump campaign adviser ...	politicsNews	December 30, 2017	1
4	Trump wants Postal Service to charge 'much mor...	SEATTLE/WASHINGTON (Reuters) - President Donal...	politicsNews	December 29, 2017	1

At this point, combining and labeling the datasets maintains traceability to the original data sources while creating a clear and uniform structure for preprocessing and model training.

5.2 Text Cleaning

Inconsistent formatting, invisible characters, and empty strings are common in raw text and are missed by the usual missing-value tests. Therefore, before tokenization and feature extraction, cleaning and normalisation are necessary to minimise noise and ensure that the dataset contains significant linguistic information. Normalisation is necessary to reduce visible variations in the text, such as variations in capitalisation, uneven spacing, and invisible characters, that do not contribute to semantic meaning. In a lack of normalisation, identical words might be regarded as separate tokens, unnecessary expanding the vocabulary and making feature representations more limited. The model can learn from consistent and similar linguistic patterns by applying text normalisation before tokenization, improving the stability of later classification performance and the reliability of feature extraction.

In [14]:
```
# check empty string values in df
(df.apply(lambda col: col.astype(str).str.strip() == "")).sum()
```

Out[14]:
```
title        0
text       631
subject      0
date         0
label        0
dtype: int64
```

In [15]:
```
# Normalisation function
def textnormalisation(s: str) -> str:
    # Check is s is empty
```

```
    if pd.isna(s):
        # return empty string if it is empty
        return ""
    # change the input into a string format
    s = str(s)
    # remove the zero-width spaces
    s = s.replace("\u200b", " ")
    # remove the spaces before and end of text
    s = s.strip()
    # change the tabs and line breaks with one space
    s = re.sub(r"\s+", " ", s)
    return s
```

In [16]:
```
# Apply normalisation for these columns
for col in ["title", "text", "subject", "date"]:
    if col in df.columns:
        df[col] = df[col].apply(textnormalisation)
```

In [17]:
```
# Remove rows when both title and text are empty. In the current data I only have t
# So this will not work but in future this might be needed for different dataset
df = df[~((df["title"] == "") & (df["text"] == ""))].copy()
```

In [18]:
```
# merge the title and text and store in content column
df["content"] = (df["title"] + " " + df["text"]).str.strip()
```

In [19]:
```
df.to_csv("cleaned_dataset.csv", index=False)
```

By addressing missing data, removing hidden characters, and removing whitespace, this preprocessing step sets up textual fields. Before all textual fields are combined and sent into the modeling pipeline, this step is carried out to make sure they are all in a consistent and readable format. Standardising the text at this point ensures that following tokenization and feature extraction work with clean, consistent inputs and stops formatting errors from spreading into the combined content field. Since every entry in the current dataset has at least one non-empty textual field, no records are deleted. In order to provide an organised text representation for further analysis, the text and title are then combined into a single content column.

5.3 Tokenization

Tokenization enables machine learning models to analyse textual patterns by breaking up continuous text into discrete tokens, usually words. Because it provides the fundamental units from which features are built, this stage is important for both frequency-based and embedding-based text representations.

In [20]:
```
# Import required library
import nltk
from nltk.tokenize import word_tokenize
```

In [21]:
```
# Download the nltk library
```

```
nltk.download("punkt")
```

```
[nltk_data] Downloading package punkt to
[nltk_data]     C:\Users\Subathra\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt is already up-to-date!
```

Out[21]: True

In [22]:
```
# tokenize function
def texttokenize(text):
    # change the text to lowercase and split it into words
    tokens = word_tokenize(text.lower())
    # remove not letters or numbers
    tokens = [re.sub(r"[^a-z0-9]+", "", t) for t in tokens]
    # revmoe empty string
    tokens = [t for t in tokens if t]
    return tokens
```

In [23]:
```
# Apply the tokenization to the df
df["tokens"] = df["content"].apply(texttokenize)
df.head()
```

Out[23]:

	title	text	subject	date	label	content	toke
0	As U.S. budget fight looms, Republicans flip t...	WASHINGTON (Reuters) - The head of a conservat...	politicsNews	December 31, 2017	1	As U.S. budget fight looms, Republicans flip t...	[as, budg fight, loon republica
1	U.S. military to accept transgender recruits o...	WASHINGTON (Reuters) - Transgender people will...	politicsNews	December 29, 2017	1	U.S. military to accept transgender recruits o...	[us, milita to, acce transgend recru
2	Senior U.S. Republican senator: 'Let Mr. Muell...	WASHINGTON (Reuters) - The special counsel inv...	politicsNews	December 31, 2017	1	Senior U.S. Republican senator: 'Let Mr. Muell...	[senior, republica senator, l mr, mu
3	FBI Russia probe helped by Australian diplomat...	WASHINGTON (Reuters) - Trump campaign adviser ...	politicsNews	December 30, 2017	1	FBI Russia probe helped by Australian diplomat...	[fbi, russ prob helped, b australi
4	Trump wants Postal Service to charge 'much mor...	SEATTLE/WASHINGTON (Reuters) - President Donal...	politicsNews	December 29, 2017	1	Trump wants Postal Service to charge 'much mor...	[trum wan post service, charg mu

By standardising case and removing non-alphanumeric characters, this tokenization process converts each article into a structured set of cleaned word tokens. Token lists that are produced offer a uniform textual representation that enables further feature extraction and modeling.

5.4 Lemmatization

Lemmatization combines many grammatical variants of the same term by reducing tokens to their base or dictionary form. This improves generalisation and decreases vocabulary size, especially for statistical models that depend on word frequency counts.

In [24]:
```
# Import the required library
from nltk.stem import WordNetLemmatizer
```

In [25]:
```
# download the nltk library
nltk.download("wordnet")
nltk.download("omw-1.4")
```

```
[nltk_data] Downloading package wordnet to
[nltk_data]     C:\Users\Subathra\AppData\Roaming\nltk_data...
[nltk_data]   Package wordnet is already up-to-date!
[nltk_data] Downloading package omw-1.4 to
[nltk_data]     C:\Users\Subathra\AppData\Roaming\nltk_data...
[nltk_data]   Package omw-1.4 is already up-to-date!
```

Out[25]: True

In [26]:
```
# variable store the WordNetLemmatizer
lemmatizer = WordNetLemmatizer()
```

In [27]:
```
# function to lemmatiza tokens
def tokenslemmatize(tokens):
    return [lemmatizer.lemmatize(token) for token in tokens]
```

In [28]:
```
# apply the lemmatization
df["lemmas"] = df["tokens"].apply(tokenslemmatize)
df.head()
```

Out[28]:

	title	text	subject	date	label	content	toke
0	As U.S. budget fight looms, Republicans flip t...	WASHINGTON (Reuters) - The head of a conservat...	politicsNews	December 31, 2017	1	As U.S. budget fight looms, Republicans flip t...	[as, budg fight, loon republica
1	U.S. military to accept transgender recruits o...	WASHINGTON (Reuters) - Transgender people will...	politicsNews	December 29, 2017	1	U.S. military to accept transgender recruits o...	[us, milita to, acce transgend recru
2	Senior U.S. Republican senator: 'Let Mr. Muell...	WASHINGTON (Reuters) - The special counsel inv...	politicsNews	December 31, 2017	1	Senior U.S. Republican senator: 'Let Mr. Muell...	[senior, republica senator, l mr, mu
3	FBI Russia probe helped by Australian diplomat...	WASHINGTON (Reuters) - Trump campaign adviser ...	politicsNews	December 30, 2017	1	FBI Russia probe helped by Australian diplomat...	[fbi, russ prob helped, b australia
4	Trump wants Postal Service to charge 'much mor...	SEATTLE/WASHINGTON (Reuters) - President Donal...	politicsNews	December 29, 2017	1	Trump wants Postal Service to charge 'much mor...	[trum wan post service, charg mu

By changing tokens to their base word forms, such as singular nouns and verbs, lemmatization reduces word variety. By treating semantically comparable word forms as a single representation, this helps in vocabulary standardisation and may reduce text feature lack. Lemmatization may encourage more stable feature extraction and enhance consistency across downstream classification techniques by reducing lexical variation before modeling.

5.5 Data Preparation for Statistical Models (TF-IDF)

Token frequency-derived fixed-length numerical vectors are necessary for statistical classifiers. TF-IDF is appropriate for challenges involving the detection of fake news because it highlights discriminative phrases while reducing the impact of extremely common words. Because it converts unstructured text into a sparse numerical feature matrix that emphasises discriminative terms while down-weighting common words, TF-IDF is used for statistical models. This representation facilitates quick training and comprehensible feature weighting, and it works well with linear classifiers.

In [29]:
```
# Import the library
from sklearn.model_selection import train_test_split
```

```
from sklearn.feature_extraction.text import TfidfVectorizer
```

In [30]:
```
# store the target and feature variable
X = df["content"]
y = df["label"]
```

In [31]:
```
# train test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_sta
```

In [32]:
```
# Convert text into numerical TF-IDF using single words and word pairs
tf_idf = TfidfVectorizer(stop_words="english",max_df=0.9,min_df=2,ngram_range=(1, 2
```

In [33]:
```
# fit train data transform test data
X_train_tf_idf = tf_idf.fit_transform(X_train)
X_test_tf_idf = tf_idf.transform(X_test)
```

In [34]:
```
# Print statements
print("X_train shape:",X_train_tf_idf.shape)
print("X_test shape:",X_test_tf_idf.shape)
```

```
X_train shape: (35918, 1225478)
X_test shape: (8980, 1225478)
```

The training and test datasets are converted into high-dimensional sparse TF-IDF feature matrices in this step, with each feature representing the significance of a word or word pair inside an article. These representations capture discriminative lexical patterns and work well with linear and probabilistic classification models that are frequently used in the detection of fake news.

5.6 Data Preparation for Embedding Models (Word Vectors)

Instead of frequency tables, embedding-based models need token sequences. Documents can be represented as dense vectors that encode contextual similarity due to word vector representations, which capture the semantic links between words.

In [35]:
```
# Import the library
from gensim.models import Word2Vec
import numpy as np
```

In [36]:
```
# convert the lemmas into list
sentences = df["lemmas"].tolist()
```

In [37]:
```
# word vector model on the lemmatise text
vectormodel = Word2Vec(sentences=sentences,vector_size=100,window=5,min_count=2,wor
```

In [38]:
```
# convert document into single vector
def documentvector(tokens, model):
    # word vectos for word the exist in the vocab
    vectors = [model.wv[t] for t in tokens if t in model.wv]
    # if not valid return 0 vector
    if not vectors:
```

```
        return np.zeros(model.vector_size)
    return np.mean(vectors, axis=0)
```

In [39]:
```
# stack the vectors into single matrix
documentvectors = np.vstack(df["lemmas"].apply(lambda x: documentvector(x, vectormo
documentvectors.shape
```

Out[39]: (44898, 100)

Instead of using token frequency to describe text, embedding-based feature generation uses dense semantic vectors that are learned from word co-occurrence patterns. Each article is mapped to a fixed-length vector that captures semantic similarity between texts by averaging word embeddings. This helps with downstream classification tasks where contextual meaning could be useful.

The text has been prepared for both frequency-based and embedding-based representations by means of local dataset conversion, cleaning, tokenization, and lemmatization. While embedding models create dense semantic document vectors using token sequences, statistical models rely on TF-IDF vectors obtained from cleaned text. This part creates a fair basis for further model training and assessment by applying representation-specific preprocessing to a shared base dataset.

6. Baseline Performance

This section assesses a baseline classification model to offer a benchmark for performance comparison, building directly on the TF-IDF feature representation and train-test split. Due to their ease of use, interpretability, and robust empirical performance, classical fake news detection studies often use linear classifiers trained on frequency-based text representations as benchmarks (Oshikawa, Qian, and Wang, 2020). Accordingly, a Logistic Regression classifier is developed as the baseline model using the obtained TF-IDF features. Since logistic regression is a common linear classifier for high-dimensional sparse TF-IDF features, it is selected as the baseline. Before introducing more complicated models, it offers a useful benchmark and trains quickly.

6.1 Baseline Model Implementation

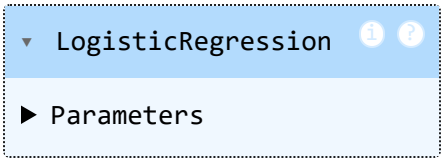
A Logistic Regression classifier is trained and assessed on the held-out test set using the TF-IDF feature matrices produced in Section 5. This model provides a clear and reliable standard by which more sophisticated methods can be evaluated.

In [40]:
```
# Import required libraries
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, precision_recall_fscore_support, classi
```

In [41]:
```
# Train Logistic Regression baseline using prepared TF-IDF features
baseline_model = LogisticRegression(max_iter=1000, random_state=42)
```

In [42]:
```
# Train the baseline model
baseline_model.fit(X_train_tf_idf, y_train)
```

Out[42]:



In [43]:
```
# Predictions
y_prediction = baseline_model.predict(X_test_tf_idf)
```

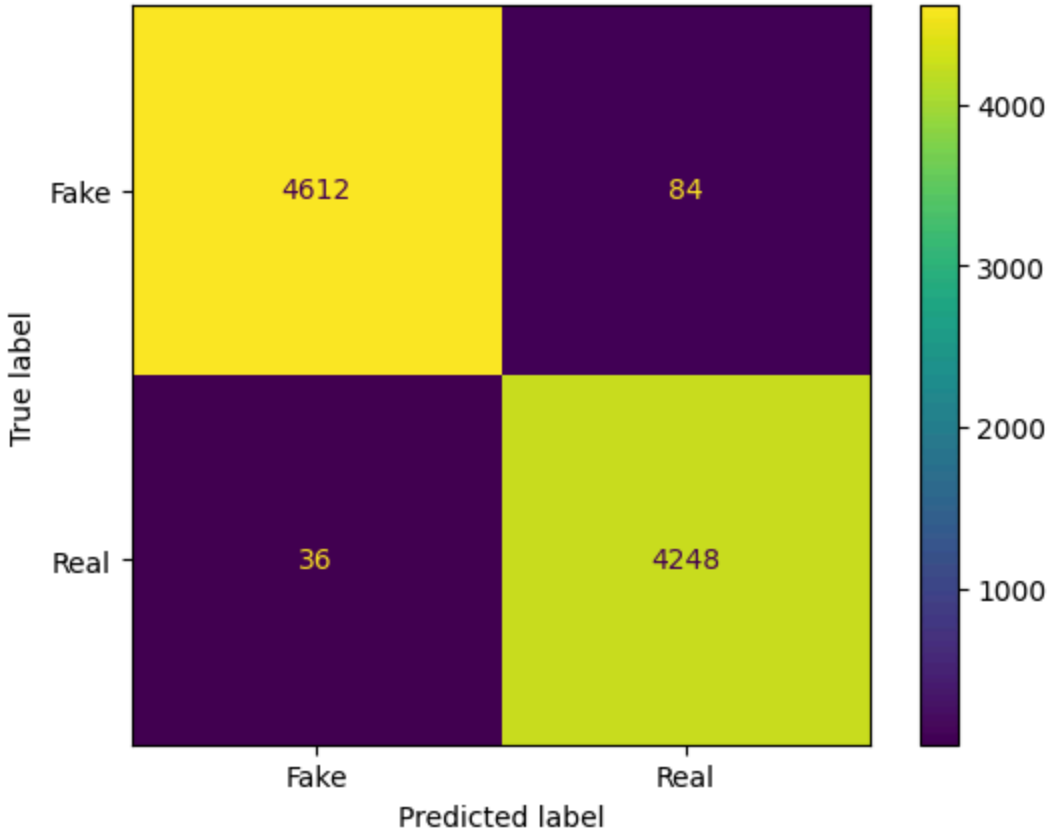
6.2 Summary Performance

Standard classification metrics, such as accuracy, precision, recall, and F1-score, are used to assess the baseline model. These measures offer a fair evaluation of both class-specific performance and overall accuracy in the binary fake news detection.

In [44]:
```
# Evaluation metrics
accuracy = accuracy_score(y_test, y_prediction)
precision, recall, f1, _ = precision_recall_fscore_support(y_test, y_prediction, av
```

In [45]:
```
# confusion matrix
cm = confusion_matrix(y_test, y_prediction)
show = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=["Fake", "Real"])
show.plot()
```

Out[45]:
```
<sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay at 0x1ac5a02b970>
```



In [46]:
```
# print statements
print(f"Baseline Accuracy:  {accuracy:.4f}")
print(f"Baseline Precision: {precision:.4f}")
print(f"Baseline Recall:    {recall:.4f}")
print(f"Baseline F1-score:  {f1:.4f}")
print(classification_report(y_test, y_prediction, digits=4))
```

```
Baseline Accuracy:  0.9866
Baseline Precision: 0.9806
Baseline Recall:    0.9916
Baseline F1-score:  0.9861
              precision    recall  f1-score   support

           0     0.9923    0.9821    0.9872      4696
           1     0.9806    0.9916    0.9861      4284

    accuracy                         0.9866      8980
   macro avg     0.9864    0.9869    0.9866      8980
weighted avg     0.9867    0.9866    0.9866      8980
```

The baseline Logistic Regression model trained on TF-IDF features shows good overall performance with an accuracy of 98.66% and an F1-score of 98.61%. This behaviour is further explained by the confusion matrix, which accurately classifies 4,248 real news and 4,612 fake news. There are few misclassifications where 36 real articles were mistakenly classed as fake (false negatives) while 84 fake articles were mistakenly labelled as true (false positives). While the high precision (98.06%) indicates that relatively few real news articles are mistakenly marked as fake, the high recall for real news (99.16%) shows that the model is

especially good at identifying genuine information. When taken as a whole, these findings show that lexical frequency information by itself is highly discriminative for this dataset, creating a strong and competitive baseline against which advanced models must show unique and measurable improvements.

7. Comparative Classification Approach

This part evaluates two different classification methods which are a traditional statistical model and a modern deep learning model, building on the baseline. The objective is to determine whether measurable improvements in performance on the fake news detection task result from improved model complexity and representational capacity. In particular, the design, training behaviour, and classification performance of a Support Vector Machine (SVM) classifier and a Long Short-Term Memory (LSTM) neural network are compared.

7.1 Traditional Statistical Model: Support Vector Machine (SVM)

Support Vector Machines are statistical classifiers that are often used for text classification problems because they work well in high-dimensional feature spaces. By maximizing the margin between classes, linear SVMs frequently outperform simple linear models when paired with TF-IDF variables, making them an effective replacement for logistic regression in the detection of fake news. Because it works well for sparse TF-IDF representations and usually does well on text classification tasks, a SVM is used. By increasing the margin between classes and enhancing generalisation, it can perform better than logistic regression.

In [47]:
```
# import required library
from sklearn.svm import LinearSVC
```

In [48]:
```
# Train SVM model
svm_model = LinearSVC(random_state=42)
svm_model.fit(X_train_tf_idf, y_train)
```

Out[48]:
```
▾ LinearSVC
▶ Parameters
```

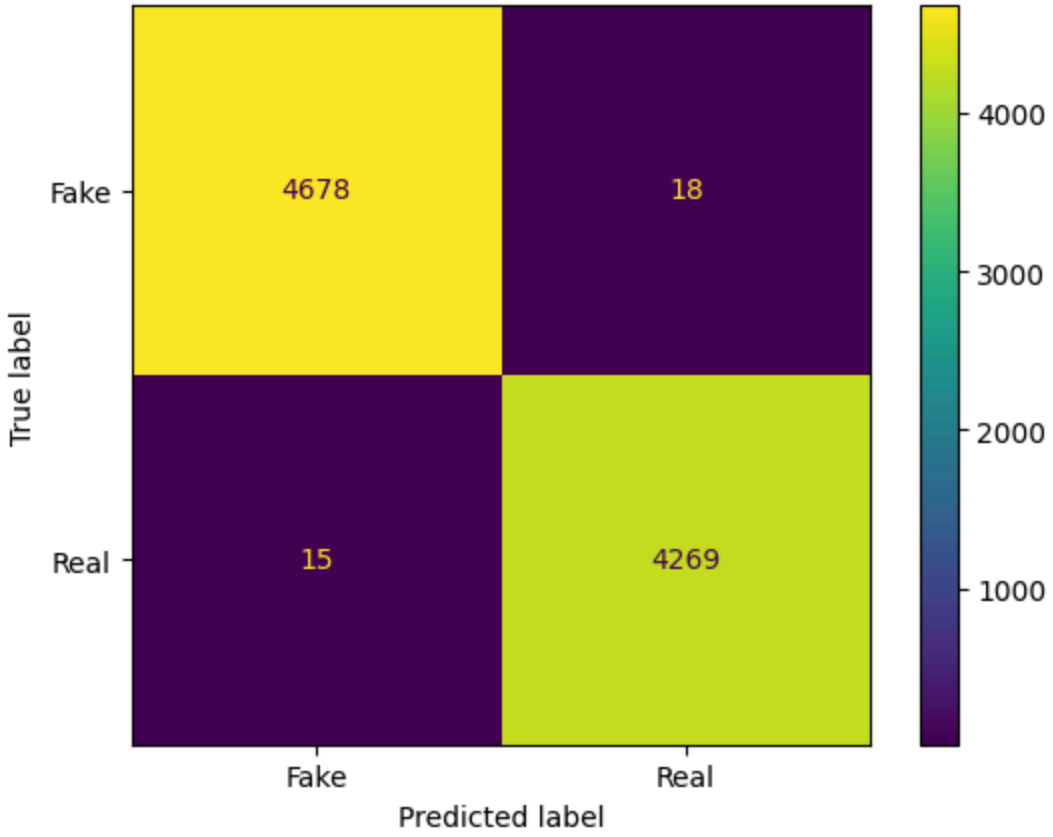
In [49]:
```
# Predictions
y_prediction_svm = svm_model.predict(X_test_tf_idf)
```

In [50]:
```
# Evaluation
svm_accuracy = accuracy_score(y_test, y_prediction_svm)
svm_precision, svm_recall, svm_f1, _ = precision_recall_fscore_support(y_test, y_pr
```

In [51]:
```
# confusion matrix
cm = confusion_matrix(y_test, y_prediction_svm)
```

```
show = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=["Fake", "Real"])
show.plot()
```

Out[51]: <sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay at 0x1ea59383eb0>



In [52]:
```
# Print statements
print(f"SVM Accuracy:  {svm_accuracy:.4f}")
print(f"SVM Precision: {svm_precision:.4f}")
print(f"SVM Recall:    {svm_recall:.4f}")
print(f"SVM F1-score:  {svm_f1:.4f}")
print(classification_report(y_test, y_prediction_svm, digits=4))
```

```
SVM Accuracy:  0.9963
SVM Precision: 0.9958
SVM Recall:    0.9965
SVM F1-score:  0.9961
              precision    recall  f1-score   support

           0     0.9968    0.9962    0.9965      4696
           1     0.9958    0.9965    0.9961      4284

    accuracy                         0.9963      8980
   macro avg     0.9963    0.9963    0.9963      8980
weighted avg     0.9963    0.9963    0.9963      8980
```

The Support Vector Machine model performs extremely well on the TF-IDF feature representation, with an overall accuracy of 99.63% and an F1-score of 99.61%. Only 15 true news articles were incorrectly labelled as fake (false positives) and 18 fake news articles were incorrectly classified as real (false negatives), according to the confusion matrix, which

reveals that 4,269 real news articles and 4,678 fake news articles are accurately classified. The consistently high precision (99.58%) and recall (99.65%) across both classes indicate this very low number of errors. These findings show the effectiveness of margin-based classifiers on high-dimensional lexical feature spaces and show that the SVM expands extremely well on this dataset despite depending only on frequency-based information, offering a highly competitive statistical benchmark for comparison with deeper embedding-based models.

7.1.1 Strengths and Limitations (SVM)

Strength:

In high-dimensional TF-IDF feature spaces, the SVM model performs well in generalisation. It usually provides dependable results with little hyperparameter modification and works well with sparse text formats. In comparison to more complex embedding-based methods, the model is computationally efficient.

Limitations:

Only lexical frequency features from the TF-IDF representation are used by the SVM. Word order and syntactic layout in news items are not specifically modelled by it. Deeper semantic or contextual relationships found in the text are not captured by the model. Thus, the quality and clarity of the selected feature representation determine how well it performs.

7.2 Deep Learning Model: Long Short-Term Memory (LSTM)

Deep learning techniques may learn sequential and contextual text representations, in contrast to statistical models. Because LSTM networks are made to recognise long-range dependencies in sequences, they can be used to model narrative structure and contextual clues that might point to false or misleading information in news items.

In [53]:
```
# Import required libraries
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from sklearn.model_selection import train_test_split
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense, Dropout
```

```
WARNING:tensorflow:From C:\Users\Subathra\anaconda3\envs\tf_work\lib\site-packages\k
eras\src\losses.py:2976: The name tf.losses.sparse_softmax_cross_entropy is deprecat
ed. Please use tf.compat.v1.losses.sparse_softmax_cross_entropy instead.
```

Text is transformed into fixed-length integer sequences for the neural model using tokenizer and padding, which are necessary inputs for an LSTM. To manage memory usage and reduce overfitting, the vocabulary size and maximum sequence length are capped.

In [54]:
```
# Tokenisation for the model
tokenizer = Tokenizer(num_words=20000, oov_token="<OOV>")
```

```
tokenizer.fit_on_texts(X_train)
```

In [55]:
```
# Convert text to sequences
X_train_sequences = tokenizer.texts_to_sequences(X_train)
X_test_sequences = tokenizer.texts_to_sequences(X_test)
```

In [56]:
```
# Maximum length of sequence
max_len = 300

# pad training sequence to have same length
X_train_pad = pad_sequences(X_train_sequences, maxlen=max_len, padding="post")
# pad testing sequence to have same length
X_test_pad = pad_sequences(X_test_sequences, maxlen=max_len, padding="post")
```

In [57]:
```
# Build the model
model = Sequential([
    Embedding(input_dim=20000, output_dim=128, input_length=max_len),
    LSTM(128),
    Dropout(0.3),
    Dense(1, activation="sigmoid")
])
model.summary()
```

```
WARNING:tensorflow:From C:\Users\Subathra\anaconda3\envs\tf_work\lib\site-packages\keras\src\backend.py:873: The name tf.get_default_graph is deprecated. Please use tf.compat.v1.get_default_graph instead.

Model: "sequential"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 embedding (Embedding)       (None, 300, 128)          2560000

 lstm (LSTM)                 (None, 128)               131584

 dropout (Dropout)           (None, 128)               0

 dense (Dense)               (None, 1)                 129

=================================================================
Total params: 2691713 (10.27 MB)
Trainable params: 2691713 (10.27 MB)
Non-trainable params: 0 (0.00 Byte)
_________________________________________________________________
```

In [58]:
```
# Compile the model
model.compile(optimizer="adam", loss="binary_crossentropy",metrics=["accuracy"])
```

```
WARNING:tensorflow:From C:\Users\Subathra\anaconda3\envs\tf_work\lib\site-packages\keras\src\optimizers\__init__.py:309: The name tf.train.Optimizer is deprecated. Please use tf.compat.v1.train.Optimizer instead.
```

An embedding layer, an LSTM unit, and a sigmoid output neuron for binary classification make up the LSTM architecture. Binary cross-entropy is chosen as the typical loss function

for two-class problems, and Adam optimization is used because of its excellent empirical performance in NLP tasks.

In [59]:
```
# Import required library
from tensorflow.keras.callbacks import EarlyStopping
```

In [60]:
```
# early stop function
early_stop = EarlyStopping(monitor="val_loss",patience=2,restore_best_weights=True,
```

Overtraining can result in overfitting, a situation in which the model keeps getting better on the training set while its generalisation performance stops getting better. Early stop was used to help this by restoring the top-performing weights and keeping an eye on validity loss. This increases efficiency and lowers the risk of overfitting by enabling the model to train up to a maximum epoch limit while automatically stopping training once validation performance stops improving.

In [61]:
```
# Train the model
history = model.fit(X_train_pad, y_train,validation_split=0.2,epochs=20,batch_size=
```

```
Epoch 1/20
WARNING:tensorflow:From C:\Users\Subathra\anaconda3\envs\tf_work\lib\site-packages\k
eras\src\utils\tf_utils.py:492: The name tf.ragged.RaggedTensorValue is deprecated.
Please use tf.compat.v1.ragged.RaggedTensorValue instead.

WARNING:tensorflow:From C:\Users\Subathra\anaconda3\envs\tf_work\lib\site-packages\k
eras\src\engine\base_layer_utils.py:384: The name tf.executing_eagerly_outside_funct
ions is deprecated. Please use tf.compat.v1.executing_eagerly_outside_functions inst
ead.

449/449 [==============================] - 82s 179ms/step - loss: 0.3352 - accuracy:
0.8445 - val_loss: 0.2973 - val_accuracy: 0.9195
Epoch 2/20
449/449 [==============================] - 85s 189ms/step - loss: 0.0917 - accuracy:
0.9761 - val_loss: 0.0887 - val_accuracy: 0.9756
Epoch 3/20
449/449 [==============================] - 84s 188ms/step - loss: 0.1171 - accuracy:
0.9656 - val_loss: 0.1287 - val_accuracy: 0.9630
Epoch 4/20
449/449 [==============================] - ETA: 0s - loss: 0.0642 - accuracy: 0.9846
Restoring model weights from the end of the best epoch: 2.
449/449 [==============================] - 80s 177ms/step - loss: 0.0642 - accuracy:
0.9846 - val_loss: 0.0905 - val_accuracy: 0.9777
Epoch 4: early stopping
```

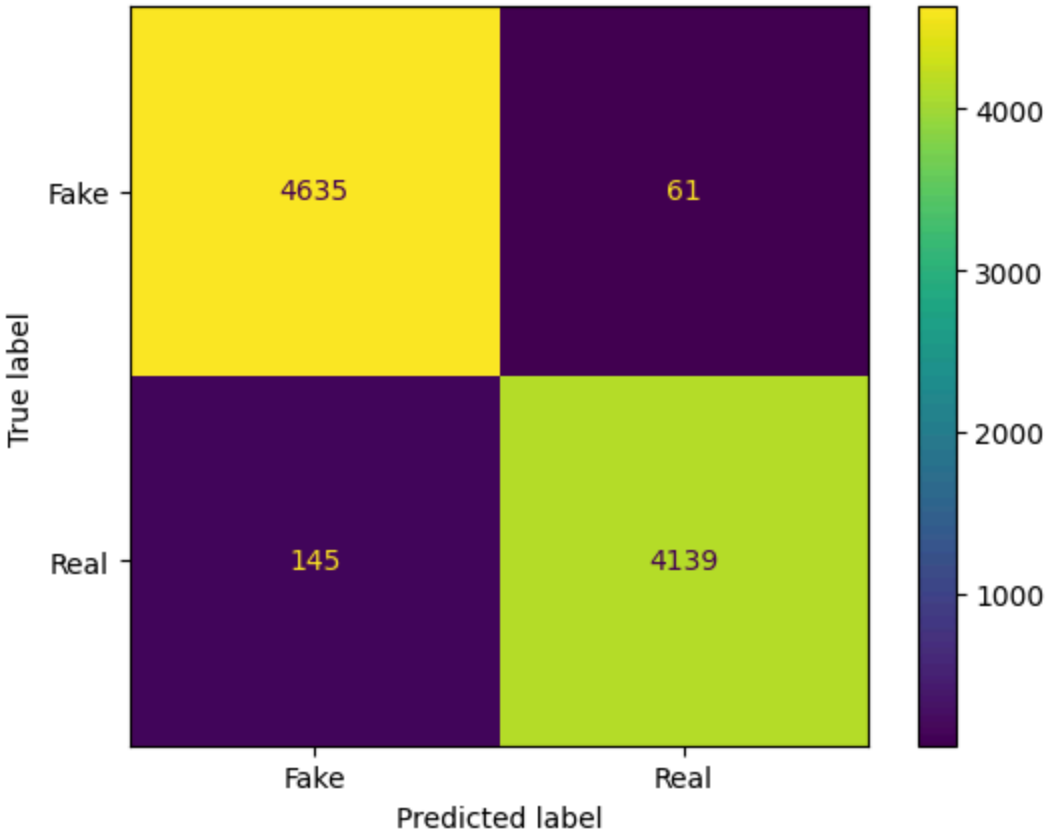
In [62]:
```
# predictions
y_prediction_lstm = (model.predict(X_test_pad) > 0.5).astype(int).flatten()
```

```
281/281 [==============================] - 9s 33ms/step
```

In [63]:
```
# metrices to evaluate
lstm_accuracy = accuracy_score(y_test, y_prediction_lstm)
lstm_precision, lstm_recall, lstm_f1, _ = precision_recall_fscore_support(y_test, y
```

In [64]:
```
# confusion matric
cm = confusion_matrix(y_test, y_prediction_lstm)
show = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=["Fake", "Real"])
show.plot()
```

Out[64]: <sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay at 0x1ea608effd0>



In [65]:
```
# Print statements
print(f"LSTM Accuracy:  {lstm_accuracy:.4f}")
print(f"LSTM Precision: {lstm_precision:.4f}")
print(f"LSTM Recall:    {lstm_recall:.4f}")
print(f"LSTM F1-score:  {lstm_f1:.4f}")
print(classification_report(y_test, y_prediction_lstm, digits=4))
```

```
LSTM Accuracy:  0.9771
LSTM Precision: 0.9855
LSTM Recall:    0.9662
LSTM F1-score:  0.9757
              precision    recall  f1-score   support

           0     0.9697    0.9870    0.9783      4696
           1     0.9855    0.9662    0.9757      4284

    accuracy                         0.9771      8980
   macro avg     0.9776    0.9766    0.9770      8980
weighted avg     0.9772    0.9771    0.9770      8980
```

With an F1-score of 97.57% and an overall accuracy of 97.71%, the LSTM-based deep learning model performs well but not as well than the top TF-IDF-based statistical models.

4,139 true news articles and 4,635 fake news articles are accurately classified, according to the confusion matrix, however 145 true articles are incorrectly classified as fake (false positives) and 61 fake articles are incorrectly classed as true (false negatives). While retaining a precision of 98.55%, which indicates a comparatively low rate of incorrect real news predictions, the model has a recall of 96.62% for real news, indicating it is effective at identifying real information. The comparable weighted average and scores support the idea that both classes performance was balanced. These findings show that the LSTM may identify sequential and contextual patterns in the text. However, in this dataset, the discriminative ability of high-dimensional lexical features used by traditional statistical models is not beaten by such contextual modeling.

7.2.1 Strengths and Limitations (LSTM)

Strength:

Frequency-based models are unable to capture contextual information and sequential dependencies seen in news content, but the LSTM model can. The LSTM can learn more complex text representations than just surface-level lexical patterns by simulating word order and time relationships.

Limitation:

In comparison to traditional statistical methods, the LSTM model brings greater computational complexity and longer training times. Sequence length, embedding size, and network depth are examples of hyperparameters that affect model performance. Larger training datasets are usually needed for deep learning models to regularly beat well-tuned statistical baselines.

7.3 Comparative Results and Analysis (SVM VS LSTM)

Model	Accuracy	Precision	Recall	F1-score	False Positives	False Negatives
SVM	0.9963	0.9958	0.9965	0.9961	18	15
LSTM	0.9771	0.9855	0.9662	0.9757	145	61

With an accuracy of 99.63% and an F1-score of 99.61%, the SVM model performs the best overall. The classification report shows that the SVM is successful in reducing both false positives and false negatives, with highly balanced precision and recall for both fake and real news classes. The effectiveness of margin-based classifiers in high-dimensional TF-IDF representations, where discriminative lexical patterns are common, is demonstrated by this performance.

With an accuracy of 97.71% and an F1-score of 97.57%, the LSTM model likewise shows outstanding performance. The model's effectiveness in detecting true articles is

demonstrated by its extremely high recall for the true news class. This shows that the LSTM can extract contextual and sequential information from news articles, which could be useful in spotting weak language signals linked to false data.

However, despite its greater architectural complexity and ability to represent word order and context, the LSTM does not beat the SVM on this dataset. The SVM consistently outperforms the other model in all evaluation measures, with a very slight performance difference. This implies that lexical frequency patterns identified by TF-IDF are highly informative for the selected dataset, reducing the marginal advantage of more in-depth context modeling.

Practically speaking, the SVM has a number of advantages over the LSTM, such as reduced computational cost, quicker training time, and improved interpretability. On the other hand, the LSTM only slightly outperforms simpler models and requires more thorough preprocessing, longer training cycles, and careful hyperparameter adjustment. For this fake news identification task, the SVM is therefore a more effective and efficient solution under the specified data conditions.

8. Programming style

This section provides an overview of the programming techniques used to ensure that the implementation is understandable, well-organised, and straightforward.

8.1 Clear Well-commented Codes

Data preprocessing, model training, and evaluation are all covered in different, logical sections of the code. To make the workflow easier to follow and to enhance readability, meaningful variable names are constantly used.

8.2 Rationale behind model choices and parameter settings

Natural language processing best practices serve as a reference for selecting models and parameters. For statistical models, TF-IDF is chosen because it works well with sparse text data, while Support Vector Machines and Logistic Regression are used because they work well with high-dimensional features and are efficient. Sequential and contextual information in reports is captured using the LSTM model, with parameter settings selected to balance overfitting risk and model performance.

8.3 Libraries or Tools used

The approach makes use of popular Python libraries, such as TensorFlow for deep learning model building, scikit-learn for feature extraction and traditional machine learning models, pandas for data handling, and NLTK for text preprocessing.

9. Performance Analysis & Comparative Discussion

9.1 Results Comparison

Model	Accuracy	Precision	Recall	F1-score
SVM	0.9963	0.9958	0.9965	0.9961
Logistic Regression	0.9866	0.9806	0.9916	0.9861
LSTM	0.9771	0.9855	0.9662	0.9757

Every model is assessed using the same held-out test set to ensure a fair comparison. The table-level results show that, although having a more complicated architecture, the LSTM performs slightly lower than the SVM, which has the best overall performance, followed by the baseline Logistic Regression.

With precision and recall values that are extremely balanced across both fake and true news classes, the SVM achieves the best accuracy (99.63%) and F1-score (99.61%). This shows that margin-based classifiers work well with high-dimensional TF-IDF feature representations.

With an accuracy of 98.66% and an F1-score of 98.61%, the baseline Logistic Regression model also shows outstanding performance. The high discriminative potential of lexical frequency features alone for this dataset is shown by this result.

With an accuracy of 97.71% and an F1-score of 97.57%, the LSTM-based deep learning model shows good performance but drops below of the TF-IDF-based statistical models. Although the LSTM effectively detect fake news, as shown by its great recall for fake news in the classification report of 98.70%, this benefit does not result in better overall performance. These result shows that contextual and sequential modeling offers little further advantage over optimised lexical techniques for this data.

9.2 Error Patterns

For both fake and true news, the SVM continuously shows high precision and recall at the class level, suggesting strong generalisation and low misclassification. For the actual news class, on the other hand, the LSTM shows a greater choice of choosing recall over precision, indicating that although it is good in capturing contextual patterns, it may be more vulnerable to false positives when detecting fake news. The baseline model, which offers a balanced but slightly less ideal trade-off, falls in between these two extremes.

9.3 Visualisation

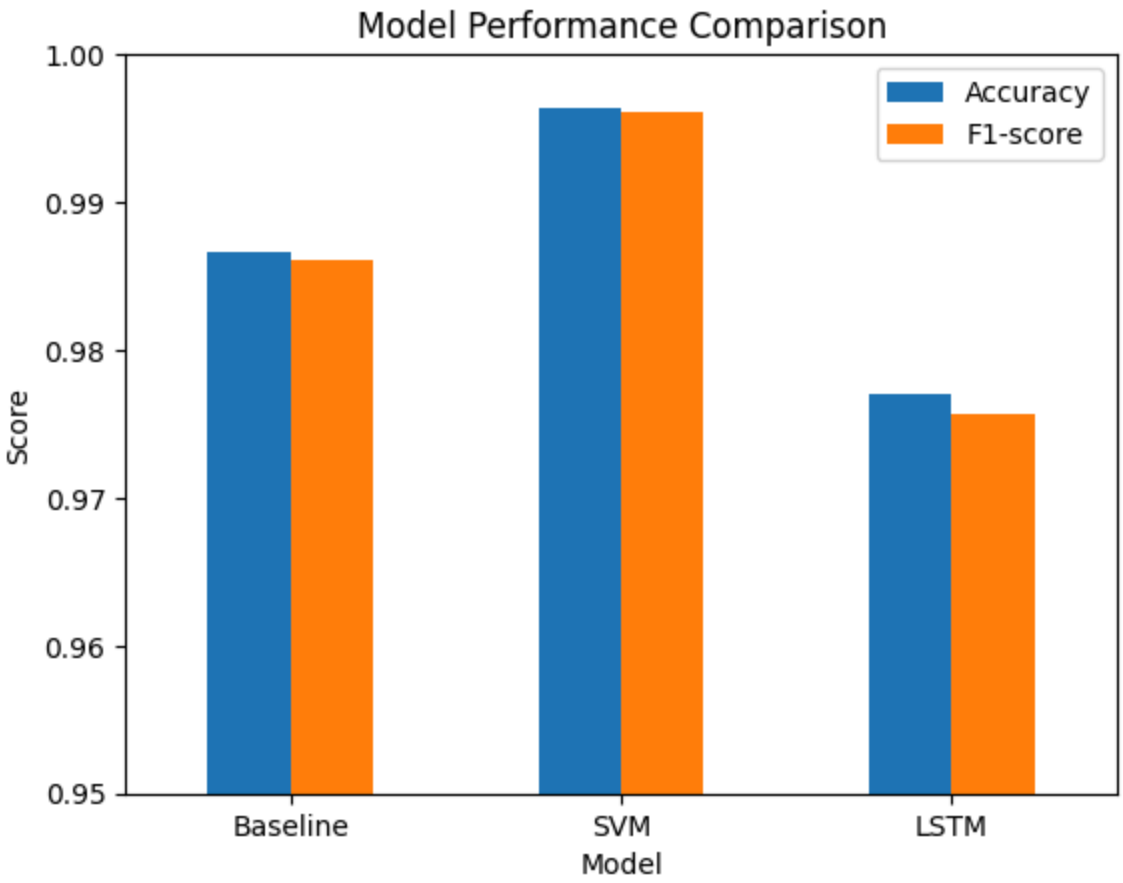
9.3.1 Bar Chart

Accuracy and F1-score are used as evaluation measures in the bar chart to provide a comparative summary of model performance. The Support Vector Machine (SVM), the LSTM-based deep learning model, and the baseline Logistic Regression model can all be directly compared using this visualisation, which highlights the respective advantages and disadvantages of each method.

In [66]:
```
# Import the required library
import matplotlib.pyplot as plt
```

In [67]:
```
# store all results in a dataframe for the plot
results = pd.DataFrame({
    "Model": ["Baseline", "SVM", "LSTM"],
    "Accuracy": [accuracy, svm_accuracy, lstm_accuracy],
    "F1-score": [f1, svm_f1, lstm_f1]
})
```

In [68]:
```
# Bar chart to compare the scores
results.set_index("Model").plot(kind="bar")
plt.title("Model Performance Comparison")
plt.ylabel("Score")
plt.ylim(0.95, 1.0)
plt.xticks(rotation=0)
plt.show()
```



The output demonstrates that the SVM achieves near-perfect classification performance by clearly outperforming the baseline and LSTM models in terms of accuracy and F1-score. Strong results are also shown by the baseline Logistic Regression model, demonstrating the efficacy of TF-IDF based statistical models for this dataset. The LSTM model, on the other hand, receives lower scores, suggesting that higher model complexity may not always result in better performance when lexical features are highly discriminative. The conclusion that statistical models are more successful and efficient for the specified fake news detection task is supported by this visualisation.

9.3.2 ROC Curve

In [69]:
```
# Import the required library
from sklearn.metrics import roc_curve, auc
```

In [70]:
```
# Probability of postives
y_score_linearregression = baseline_model.predict_proba(X_test_tf_idf)[:, 1]
```

In [71]:
```
# decision function scores for svm
y_score_svm = svm_model.decision_function(X_test_tf_idf)
```

In [72]:
```
# probabilities of positive
y_score_lstm = model.predict(X_test_pad).ravel()
```

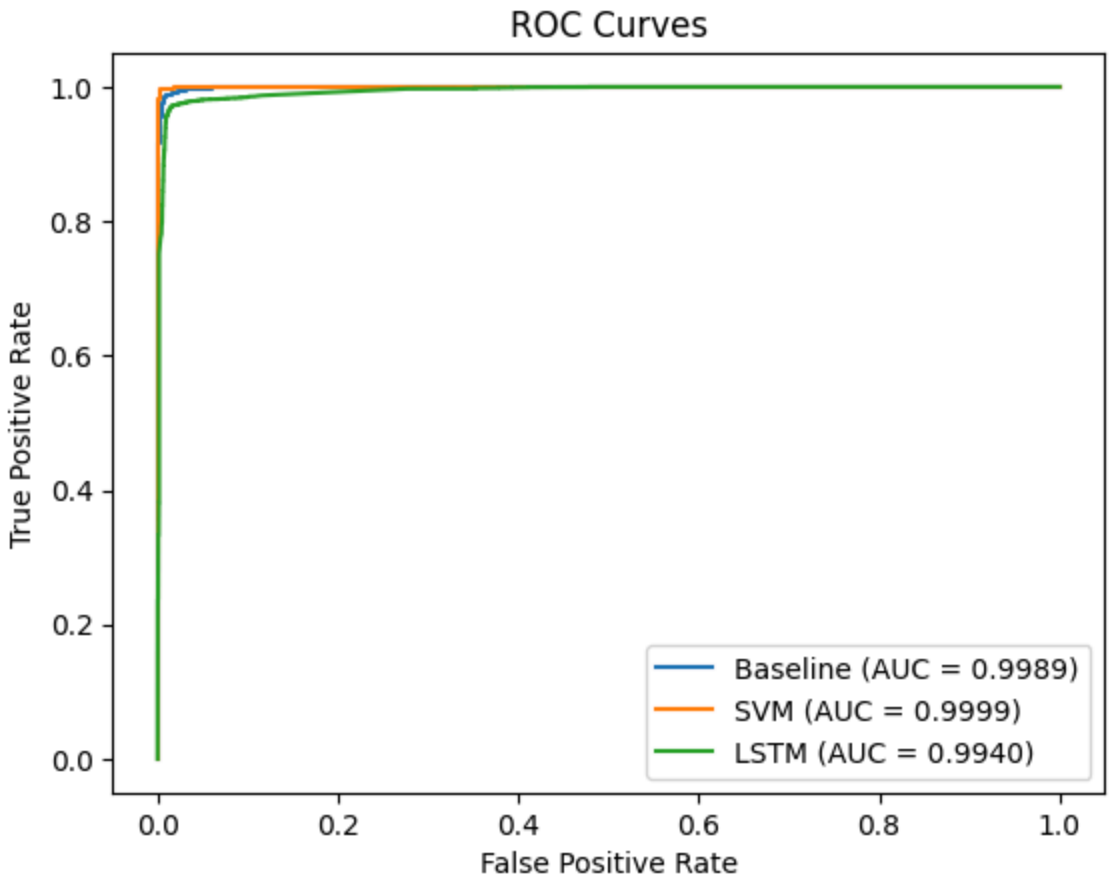
```
281/281 [==============================] - 10s 35ms/step
```

In [73]:
```
# calculate false positive rate and true positive rate and auc for logistic regress
fpr_linearregression, tpr_linearregression, _ = roc_curve(y_test, y_score_linearreg
auc_linearregression = auc(fpr_linearregression, tpr_linearregression)
```

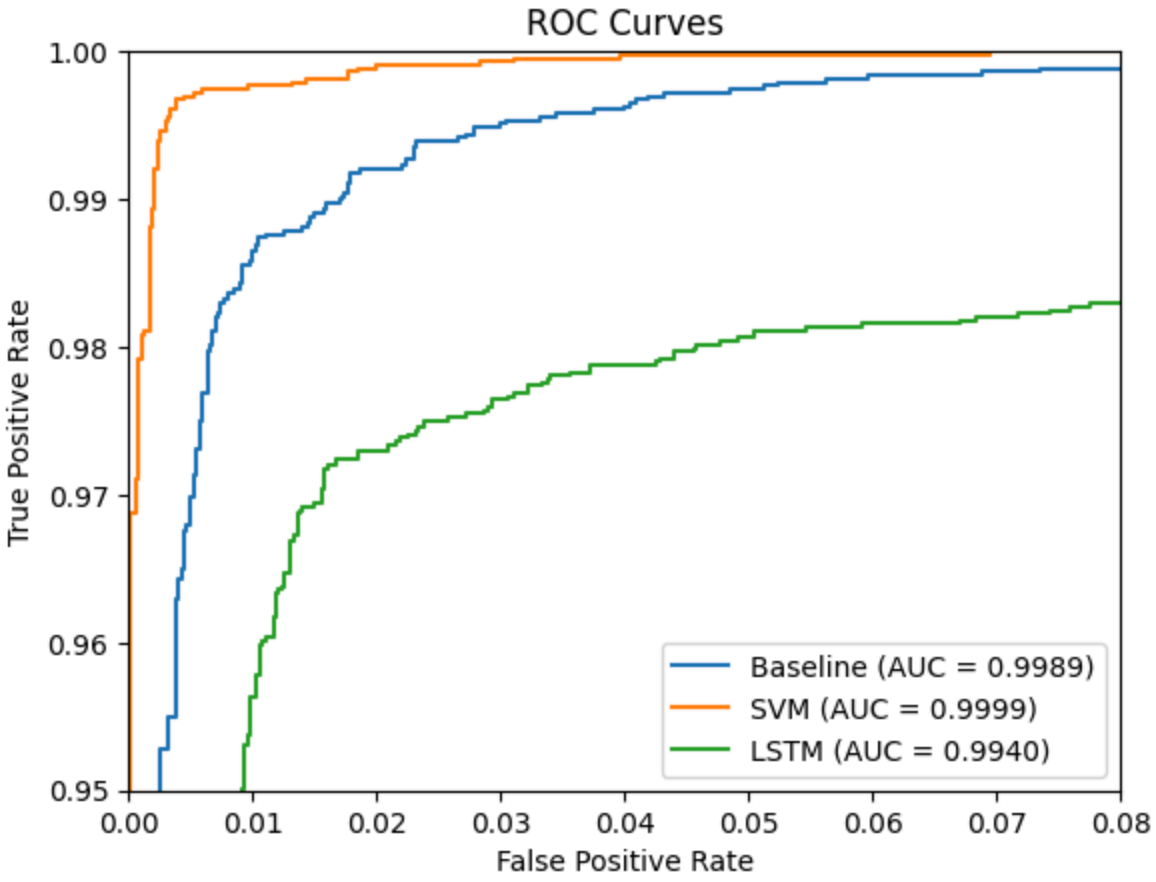
In [74]:
```
# calculate false positive rate and true positive rate and auc for svm
fpr_svm, tpr_svm, _ = roc_curve(y_test, y_score_svm)
auc_svm = auc(fpr_svm, tpr_svm)
```

In [75]:
```
# calculate false positive rate and true positive rate and auc for lstm
fpr_lstm, tpr_lstm, _ = roc_curve(y_test, y_score_lstm)
auc_lstm = auc(fpr_lstm, tpr_lstm)
```

In [76]:
```
# Roc curve with the limits
plt.plot(fpr_linearregression, tpr_linearregression, label=f"Baseline (AUC = {auc_l
plt.plot(fpr_svm, tpr_svm, label=f"SVM (AUC = {auc_svm:.4f})")
plt.plot(fpr_lstm, tpr_lstm, label=f"LSTM (AUC = {auc_lstm:.4f})")
plt.title("ROC Curves")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.legend(loc="lower right")
plt.show()
```



In [77]:
```
# Roc Curve with limits
plt.plot(fpr_linearregression, tpr_linearregression, label=f"Baseline (AUC = {auc_l
plt.plot(fpr_svm, tpr_svm, label=f"SVM (AUC = {auc_svm:.4f})")
plt.plot(fpr_lstm, tpr_lstm, label=f"LSTM (AUC = {auc_lstm:.4f})")
plt.title("ROC Curves")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.legend(loc="lower right")
plt.xlim(0, 0.08)
plt.ylim(0.95, 1.0)
plt.show()
```



The ROC curves for the assessed models are shown in plot, where discriminative differences are highlighted by zooming in on the low false positive rate region. Throughout this important range, the SVM continuously achieves the greatest true positive rate, showing its better early detection capabilities. While the LSTM shows a slower increase in true positive rate at low false positive rates, Logistic Regression performs similarly but stays somewhat below the SVM. This behaviour supports the quantitative findings and demonstrates that, for this dataset, high-dimensional lexical characteristics allow for more robust differentiation than sequence-based modeling.

9.4 Advantages and Disadvantages of Statistical vs Embedding-Based Models

Statistical Models:

Advantage:

On this dataset, statistical models perform very well empirically. The SVM has the highest accuracy and F1-score of all the models that were assessed. These models are appropriate for large-scale or real-time deployment due to their computational efficiency and short training times. Compared to deep learning models, statistical approaches are simpler to comprehend, debug, and maintain due to the modeling pipeline's simplicity. The good baseline and SVM performance show that effective results may be obtained with little hyperparameter modification.

Disadvantage:

Deeper semantic connections within text are not explicitly modelled by statistical models, which mostly rely on surface lexical patterns. They are unable to capture discourse-level structure or long-range interdependence due to their lack of sequential knowledge. Generalisation can suffer when applied to content from different times, and performance may be dependent on dataset specific features.

Embedding-Based Models:

Advantage:

By processing text as ordered sequences, embedding-based algorithms are able to extract sequential and contextual information from news. Because meaning is represented in a continuous embedding space rather than discrete frequency counts, these models are more appropriate for processing paraphrased varied text. In theory, LSTM networks can more successfully generalise to challenges where classification heavily relies on semantic nuance.

Disadvantage:

Despite being more sophisticated, the LSTM model performs worse than the statistical SVM, suggesting that sequential modeling does not offer a performance advantage for this dataset. Compared to statistical methods, embedding-based models need more processing power and take longer to train. Implementation complexity is increased by the need for additional preprocessing processes and careful hyperparameter adjustment. In applications that need explainability or transparency, the less interpretable nature of model decisions may be a drawback.

9.5 Practical Implication and Model Selection

9.5.1 Scenerios statistical models preferred

According to the observed results, when the dataset contains significant lexical signals and accuracy, speed, and simplicity are priorities, TF-IDF with SVM is the best option for large-scale false news identification. It is appropriate for production pipelines that need to be effective, repeatable, and simple to maintain, particularly when quick iteration is needed when computational resources are limited.

9.5.2 Scenerios embedding-models preferred

When an issue involves more semantic complexity than keyword separability, including paraphrased misinformation, adverse rewriting, cross-topic generalisation, or multilingual content, embedding-based models like LSTMs might be the better choice. They are also helpful in situations when TF-IDF representations could lose crucial information, such as when one anticipates that word order and context will convey significant fake clues such as stance manipulation, subtle hedging, or narrative consistency.

9.5.3 Hypothesize reasons for observed performance disparities

The findings shows that this dataset probably contains highly discriminative lexical patterns that TF-IDF features successfully capture, allowing linear classifiers to distinguish classes with extremely low error. On the other hand, since frequency-based signals can already perform the task, the LSTM might not benefit as much from modeling sequence information. Hyperparameter sensitivity such as fixed sequence length truncation, vocabulary limits, limited epochs and the possibility of learning less stable representations when simpler linear decision boundaries are adequate may further limit the LSTM's performance.

10. Project Summary and Reflections

10.1 Learning experience

The practical understanding of how natural language processing techniques are used in actual machine learning systems was supported by this study. The importance of thorough preprocessing, suitable feature representation, and methodical evaluation was brought to light when designing an end-to-end text classification pipeline. Acknowledging the importance of robust baseline models and rigorously evaluating whether increased model complexity results in significant performance gains were important learning gains for me.

10.2 Comparative Evaluation of Models

A standard preprocessing pipeline, a fixed train-test split, and the same evaluation metrics for every model were used to ensure a fair comparison. The best results were obtained by statistical classifiers that used TF-IDF features, especially Support Vector Machines, in terms of accuracy, F1-score, ROC-AUC, and confusion matrix analysis. The strong performance of logistic regression further showed the excellent discriminative ability of lexical frequency characteristics. Despite having a greater computing cost, the LSTM model did not outperform the statistical baselines despite capturing sequential information, this suggests a limit effect caused by the dataset's strong lexical signals.

10.3 Practicality and Real world Applications

Statistical models turned out to be more useful from a deployment standpoint because of their interpretability, quick training, and minimal processing requirements. They are ideal for

large-scale applications like content moderation, news monitoring, and decision-support systems because of these qualities. Although embedding-based models might be useful in situations where language is complicated or changing, their extra complexity needs to be supported by proven performance improvements.

10.4 Reproducibility and Transferability

By clearly separating the preprocessing, feature extraction, training, and evaluation phases, the modular pipeline design promotes reproducibility. With minimal changes to domain-specific data features, the method can be applied to various text classification domains such as sentiment analysis, spam detection, and topic categorisation.

10.6 Improvements and future research direction

To enhance contextual understanding, future research can investigate transformer-based architectures and pretrained language models. To improve transparency and confidence in automated false news detection systems, next directions include assessing robustness under domain change, looking into hybrid feature representations, and using explainability techniques.

References

Bisaillon, C. (2023), Fake and Real News Dataset.(Online) Available at: https://www.kaggle.com/datasets/clmentbisaillon/fake-and-real-news-dataset?select=True.csv (Accessed on: 10th December 2025)

Roumeliotis, K.I., Tselikas, N.D. and Nasiopoulos, D.K. (2025) Fake news detection and classification: A comparative study of convolutional neural networks, large language models, and natural language processing models. (Online) Available at: https://www.mdpi.com/1999-5903/17/1/2 (Accessed on: 11th December 2025)

Prachi, N.N., Habibullah, M., Rafi, M.E.H., Alam, E. and Khan, R. (2022) Detection of fake news using machine learning and natural language processing algorithms. (Online) Available at: https://www.researchgate.net/publication/365857538_Detection_of_Fake_News_Using_Machine_ (Accessed on: 11th December 2025)

Oshikawa, R., Qian, J. and Wang, W.Y. (2020) A survey on natural language processing for fake news detection. (Online) Available at: s https://arxiv.org/abs/1811.00 (Accessed on: 12th December 2025)

Probierz, B., Stefański, P. and Kozak, J. (2021) Rapid detection of fake news based on machine learning methods. (Online) Available at: 70 https://www.sciencedirect.com/science/article/pii/S187705092101 (Accessed on: 13th December 2025)

Ferguson, A.G. and Avornu, C. (2022) Fake news detection using NLP with LSTM. (Online) Available at: 97X https://irjaes.com/wp-content/uploads/2022/10/IRJAES-V7N4P54Y2 (Accessed on : 13th December 2025).pdf